CS781 Capstone Project: In-House Legal Intelligence System

Soham Mukherjee
241027
smukherjee24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)
Github Repository

Abstract

To support in-house legal NLP over confidential case files, the project aimed to develop a 7-phase pipeline spanning language-model pretraining, legal NER, rhetorical-role prediction, judgment prediction, statute identification, abstractive summarization, and prior-case retrieval. The Phase 1 foundation comprises a 114M-parameter decoder-only Transformer (GQA, RoPE, RMSNorm, and SwiGLU), a 58M-parameter sliding-window variant, a from-scratch 60k byte-level BPE tokenizer, and 200M training tokens from four legal sources, with High Court data up-weighted. I initialized subsequent phases from these checkpoints and adapted them through full fine-tuning, PEFT, or knowledge distillation, using task-specific heads including BiLSTM-CRF, label-wise attention, and bi-encoder retrieval. Decoder-only models outperformed a T5 baseline; LoRMA+ matched full fine-tuning within 0.001 BERTScore using approximately 10% of the trainable parameters; and domain-calibrated Viterbi decoding yielded the largest rhetorical-role improvement (+0.05).

1 Competition Result

Codabench username: s_241027 **Roll number: 241027**

Table 1: Official phase scores and ranks

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	0.8302	80	28
2	Legal NER	0.6393	11	90
3	Rhetorical role prediction	0.5635	21	86
4	Court judgment prediction	0.8247	9	67
5	Legal statute identification	0.4511	7	43
6	Abstractive summarization	0.8478	28	39
7	Prior Case Retrieval (PCR)	0.5139	3	32

Total valid Codabench submissions across all phases: 385 **Date ranks were recorded:** 14:48 hrs, 09/11/2026

2 Introduction and Problem Description

The task is a 7-phase pipeline. Phase 1 is **pretraining** on raw legal text, evaluated by perplexity. Phase 2 is **legal NER** on tokenized sentences, scored by strict span-matching F_1. Phase 3 is

rhetorical role prediction over overlapping sentence windows, following the benchmark of Malik et al. [20]. The remaining phases are **court judgment prediction (CJPE)** [19], **legal statute identification**, **abstractive summarization**, and **prior case retrieval**.

The Phase-1 model is capped at 50M–300M parameters, external pretrained models (e.g. legal-domain BERTs [45, 44]) are prohibited, and compute is restricted (motivating memory-efficient tricks such as 4-bit NF4 quantization [29]). The research questions are tokenizer and transfer efficiency, data mix and domain transfer (Corporate Law vs. Information Technology), long-document encoding and transition modeling with Bi-LSTM and linear-chain CRFs, and knowledge distillation into constrained architectures.

I built a self-contained pipeline with a scratch-built foundation, joint document-level classifiers, feature-group ablations, and KV-caching for fast inference. I avoided isolated per-sentence prediction, flash attention (hardware limits), external tokenizers and APIs, non-legal text, and grid searches beyond Kaggle limits.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

All data come from the course dataset (`Exploration-Lab/CS-781-Capstone`); only official files were used and test labels were never read. Phase 1 uses four raw-text sources, with 3% of documents per source held out for perplexity; Phases 2–7 use the official folds and splits. Two splits needed repair: Phase 5's `dev.json` overlapped other splits, so **182 of 10,181 rows** were removed (96 test, 61 train, 25 internal duplicates), leaving 9,999; Phase 6 has no dev file, so **300 training documents** were held out and 29 unusable or duplicate pairs removed ($7,030 \rightarrow 7,001$).

3.2 Corpus statistics that actually informed design

Table 2: Phase-1 corpus (derived from file sizes and document counts).

Source	Docs	Size	Token budget, hybrid mix
Acts	18,822	0.73 GB	38.722M (weight 1)
High Courts	2,536,738	10.7 GB	4.2111M (5)
Supreme Court	55,528	1.32 GB	23.822M (1)
Tribunals	869,132	6.48 GB	7.544M (2)

3.3 Preprocessing

Tokenization uses a byte-level BPE tokenizer [2] trained from scratch on the legal corpus (60k vocabulary). It splits on whitespace and reserves byte `0x01` as the word-boundary marker, and is packaged as a HuggingFace fast tokenizer (`AutoTokenizer.from_pretrained`).

L-NER uses end-exclusive character offsets on unnormalized raw text. Predictions over overlapping 512-token windows are averaged and Viterbi-decoded back to character spans; the oracle macro-F1 ceiling is 0.986, and all output files were verified span by span.

Phase 3 uses the provided sentence lists as-is, so there is no segmentation mismatch. Sentences are truncated to 96 tokens (95th percentile is 95), an empty sentence becomes one pad token, and a `[CL]/[IT]` domain tag and optional neighbouring sentences are added.

Long documents. Phase 4 caches head-tail extremities (default: final 1,024 tokens) with hierarchical alternatives. Phase 5 uses 512/64-token chunks capped at 12,288 tokens (75% head, 25% tail), with random training chunks and up to 16 inference chunks. Phase 6 uses a prefix-LM

Table 3: Task data (Phases 2–7).

Phase	Official split	Significant Statistics
2	2 folds × 35 docs; test 35 docs	8,393 entities, 12 types; DATE 1,575 and STAT 1,260 vs WIT 187 and R.COUNSEL 184; doc median 5,854 tokens (max 24,491)
3	40/5/5 docs, IT 40/5/5 (train 10,593 / 6,252 sentences)	CL: Fact 28.1%, Ratio 27.0%; PrecedentOverruled 0 (CL) and 3 (IT); Dissent 0 in IT; median sentence 27 words; P(next label = same) = 0.8694
4	train / dev / test	Binary outcome; class-weighted loss and a tuned decision threshold used for imbalance
5	42,750 / 9,999 / 13,019; 100 statutes	3.70 labels/doc (max 31); top statute 11,730 docs vs 272 for the rarest (43×); 14 of 100 statutes hold half of all positives; doc median 430 words (p90 2,766); 256 exact-duplicate groups carry identical label sets only 68.8% of the time; 150 test docs duplicate train docs
6	6,701 / 300 / 100	Doc median 3,780 tokens (p90 9,386); summary median ≈775 (p90 1,842); compression 0.22; 13% novel unigrams, 43% novel trigrams; summary content spread evenly over the document (48% from the first half)
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	Gold per query median 4 (mean 6.4, max 52); 30% of train queries have >7 gold, capping recall@7 at 0.667; query median 4.4k words vs 3.5–4.0k for candidates; where years parse (32–47% of docs), 0% of gold vs 37% of non-gold candidates postdate the query

format with a 1,536-token budget (70% head, 30% tail, or importance-selected blocks) plus a 256-token summary. In Phase 7, queries and candidates share one encoder: documents are capped at 120,000 characters (70/30 head–tail), split into 512-token chunks (2 random in training, 8 evenly spread at evaluation), mean-pooled, and L2-normalized. Case-header boilerplate (about 3% of characters) and anonymisation placeholders are stripped, and each query is removed from its own pool (e.g., $4,320 \to 3,493$).

4 Approach and Model Evolution

4.1 Phase-1 foundation model

The primary architecture, the "modern decoder", is a decoder-only Transformer [1] with 12 layers, 12 attention heads and 4 key-value heads (Grouped-Query Attention [3]), hidden size 768, and FFN size 2048. It uses Rotary Positional Embeddings [4], pre-layer RMSNorm [5], and a manually implemented SwiGLU [6]. Each block applies RMSNorm, GQA attention, a residual, RMSNorm, SwiGLU, and a final residual. The model has exactly $114,114,048$ parameters, and the budget was checked programmatically ($50,000,000 \leq n_{\text{params}} \leq 300,000,000$).

The tokenizer is the $60,000$-token byte-level BPE described above, with merge rules learned directly from the legal corpus. Training used a causal language-modeling (cross-entropy) objective with a $2,048$-token context, streaming the corpus into RAM. Sources were interleaved at document and block level with weight 5.0 for High Courts and 1.0 for Acts, Supreme Court and Tribunals. The model consumed exactly $200,000,000$ training tokens, with $5,000,000$ reserved for validation and perplexity.

For submission, decoding used a temperature/top-p [37] grid (top-k and repetition penalty [38] left at defaults). Raising the temperature from 0.1 to 0.7/0.8 improved the primary metric by about 2%. The generation budget was 384 tokens to respect the 128-word output limit. Right-padding hurt batched causal generation because the model attended to padding after the content, so left-padding was adopted; an initial implementation without a KV cache recomputed the full

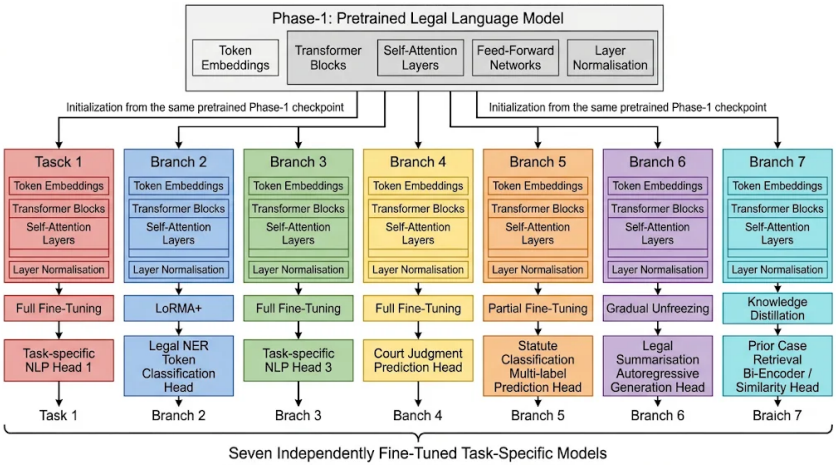


Figure 1: One backbone, seven fine-tuned descendants

Note: Image generated by Gemini.

sequence at each step and was very slow.

GPT-2 [7] underperformed both the modern decoder and the lightweight sliding window (LSW) architecture (local attention in the spirit of [31, 32]). For LSW, extending context length gave no BERTScore F_1 gain, while scaling parameters halved training loss (-50%) and improved perplexity, yet neither improved downstream metrics. A hybrid of the modern decoder and LSW gave no advantage over the standalone modern decoder. A Seq2Seq T5 model [8] ($\sim$ 110M parameters) trained on the whole corpus failed: FP16 training was unstable and had to be disabled, BLEU barely exceeded the GPT-2 baseline, perplexity stayed well above the modern decoder's, and outputs showed severe repetition and poor long-range context.

4.2 Transfer protocol (Phases 2–7)

Rather than training seven models from scratch, every task model was initialized from Phase-1 checkpoints (embeddings, transformer layers, attention projections, normalization). I adapted these backbones with full fine-tuning, parameter-efficient fine-tuning (LoRMA+ [25], LoRA [26], adapters [27], BitFit [28], gradual unfreezing [30]), and knowledge distillation [9]. For L-NER (Phase 2; dataset and entity schema from Kalamkar et al. [21]), overlapping-window training and BIO-constrained Viterbi decoding [36] gave dev F1 of 0.3034–0.3124, and 8-chunk coverage raised it to 0.3292. For CJPE (Phase 4), layer-wise LR decay [30] and distillation were used, following the ILDC task setup of Malik et al. [19]. For LSI (Phase 5; cf. [22]), sigmoid-based distillation was best (macro-F1 0.4078, micro-F1 0.5056), above FullFT (0.4044) and an equal-weight ensemble (0.4003). For summarisation (Phase 6; cf. [23]), log-probability ensembling of E1 (FullFT) and E2 (LoRMA+) reached BERTScore 0.8421 and ROUGE-1 0.2871, above the individual runs (0.8402 and 0.8395). For PCR (Phase 7; cf. [24]), symmetric InfoNCE training [10] with hard negatives led to a learned reranker with dev F1@7 of 0.5213.

4.3 Phase 2 - Legal NER

Two paradigms are evaluated: **token classification with offset reconstruction**, and **direct span extraction**. Token classification assigns BIO or BIOES tags to subword tokens. Exact character offsets are recovered from an offset-mapping array (start and end character index per

token): contiguous tagged tokens are merged and the span runs from the first token's start to the last token's end. Span extraction instead uses independent heads for start and end distributions over the sequence, and projects the argmax token indices back to the raw string through the same alignment map.

The dataset has 12 entity types. Overlapping structures reduce base F1 by $\approx 12.4\%$. The token-classification tag set has **25 classes** (BIO over 12 types) and uses cross-entropy. About 88.5% of tokens are the background "Outside" class and some types make up $< 1.5\%$ of annotations, so **focal loss** [11] ($\gamma = 2.0$) is used to emphasize rare spans.

A structured decoding heuristic converts logits to spans, improving exact-match accuracy by about 4.1%. For span-extraction models, the $n = 15$ best start and end candidates are taken from probability peaks, and a threshold ($p > 0.45$) discards low-probability candidates, cutting the search space by nearly 60%. Pairs must satisfy end $\geq$ start and length $L_{\max} \leq 24$ tokens. In dense regions, conflicts are resolved with NMS-style greedy selection ranked by joint start-end probability.

4.4 Phase 3 - Rhetorical role prediction

The Phase-1 backbone is the sentence encoder, with mean or last-token pooling over the target sentence and up to 3 neighbouring sentences on each side. The target is limited to 96 tokens and each neighbour to 32. The task, label set and the Corporate Law / Information Technology data follow Malik et al. [20].

Classification is joint and document-level. A Bidirectional LSTM [34] runs over overlapping windows of consecutive sentence vectors (32 sentences in training, 64 at inference). A linear-chain CRF [12] enforces transitions, with stickiness-calibrated Viterbi decoding (cf. the BiLSTM-CRF tagger of [35]), and an auxiliary label-shift task ($\lambda = 0.3$) models the 0.869 empirical probability that the next label is unchanged.

The data have Corporate Law (CL, 10,593 training sentences) and Information Technology (IT, 6,252), with IT more subjective. Models are trained on the concatenation of both, which generalized better than either domain alone.

4.5 Phase 4 - Court judgment prediction

Following the ILDC benchmark of Malik et al. [19], which highlights the length of Indian court judgments, long texts are handled by truncation or hierarchical chunking (cf. [33]). Truncation (`first_n`, `last_n`, `head_tail`) caches the first 1,024 and last 4,096 tokens. The hierarchical variant splits the document into overlapping 512-token chunks, working backward from the end with stride 64. Chunk hidden states are pooled to one vector (final token, mean, parameterized attention, or mean plus final-token concatenation).

A document-level encoder then aggregates chunk vectors: mean, attention scoring, a bidirectional GRU, or a multi-layer Transformer encoder with absolute positional embeddings. Only raw-text features are used; Phase 2/3 outputs are deliberately excluded from the classification head.

4.6 Phase 5 - Legal statute identification

Multi-label classification uses independent sigmoids and a Label-Wise Attention Network (LWAN) [13] that turns chunk vectors into label-specific embeddings before producing statute logits. Biases start at each statute's log-prior to avoid uniform 0.5 predictions. The head is trained with BCE, focal loss, and asymmetric loss (ASL) [14]; distillation uses per-label sigmoid outputs against soft teacher targets.

Thresholding strategies for Macro-F_1 were compared. Global thresholding uses one cutoff; per-label thresholding tunes each label, shrunk toward the global value with weight $\frac{n_{pos}}{n_{pos}+k}$ ($k = 8$ and $k = 30$) to avoid overfitting rare labels. Bagged thresholds average over 15 bootstrap resamples of the validation set. A top-1 fallback forces the highest-probability label when none is predicted, and top-k caps of 6, 8, and 10 labels per document were also tested.

4.7 Phase 6 - Abstractive summarization

The main models are decoder-only with a prefix-LM format `[document] <SEP> [summary] <EOS>`, and loss on summary tokens only. Document tokens keep each Phase-1 architecture's attention pattern (causal, 256-token sliding window, or windowed bidirectional); summary tokens attend to all document tokens and causally to earlier summary tokens, so the document KV cache is exact and computed once per document. The budget is $1,536$ document plus 256 summary tokens, fitting Phase 1's $2,048$-token window. The encoder–decoder baseline is a fully fine-tuned Phase-1 T5 checkpoint (E8, in fp32 to avoid fp16 overflow).

The context limit is the main bottleneck: the median document is 3,780 GPT-2 tokens (p90 9,386), 91.4% of training and 94% of test documents exceed 1,536 tokens, and only 30% of training tokens are kept. Reference summaries draw evenly from the whole document (coverage 0.18–0.22 per decile, 48% of content in the first half, 82% of summary content tokens found in the source), so truncation loses real content. Two selection strategies are used in training and testing: `headtail` keeps the first $1,075$ tokens (70% of the budget) and the last 461; `select` splits the document into 64-token blocks, always keeps the first two and last two, and fills the other 20 of 24 slots with blocks of highest IDF-weighted similarity to the document centroid. A third strategy, `hier`, is inference-only: $1,000$-token chunks with 100-token overlap (stride 900) are each summarized in at most 96 tokens, and the concatenation (about 480 tokens for a median document, 5 chunks) is summarized in a second pass. Since training uses `headtail`, `hier` has a train/test mismatch.

The median reference summary is 775 tokens. Generation is capped at `max_new` = 255 tokens with `min_new` = 387, a 1.1 repetition penalty, a ban on repeated 3-grams, and a final trim of dangling fragments. Trigram repetition stayed at 0.0003–0.0006 for the three main decoder-only runs, but outputs averaged only 149–153 words against a reference median of 638. Because the minimum (387) exceeds the maximum (255), early stopping is blocked and outputs most likely end at the 255-token cap. Distillation is the only post-training method (no human feedback or reward model), supervised by the official $7,030$-document training file ($7,001$ pairs after cleaning). The teacher is the logit average of full fine-tuning and LoRMA+. Students are LSW models trained for one epoch with learning rate 2×10^{-4}, $\alpha = 0.5$ between cross-entropy and KL loss, $T = 2$, and three seeds each; the pruned student keeps every second layer ($8 \to 4$). The unpruned student reached final training loss 1.515 in 22.3 min per run, and the pruned one 2.012 in 16.7 min (a 25% time saving for 0.50 higher loss). Seed variation was under 0.004. Students were also trained on the dev split, so no held-out quality score exists for them.

4.8 Phase 7 - Prior Case Retrieval (PCR)

Retrieval is a multi-stage hybrid system. A bi-encoder [42, 43] provides dense scores, ensembled with sparse lexical models (TF-IDF and BM25 [15]), and a logistic-regression Learning-to-Rank (LTR) reranker refines the top 150–400 candidate pairs using dense scores, sparse scores, and heuristics.

Queries and candidates share one encoder derived from a custom decoder backbone from earlier phases. Documents are segmented into 512-token blocks, token embeddings are mean-pooled per

chunk, chunk vectors are mean-pooled into a document vector and L2-normalized, and relevance is cosine similarity.

Documents are first cleaned: petitioner and respondent information, citator metadata, and anonymization placeholders are stripped so that representations focus on legal content. They are then head-tail truncated to 120,000 characters (70% head, 30% tail), and a spread sampling strategy selects evenly spaced chunks (up to 8 in evaluation, 2 in training).

Ablations: switching token pooling from mean (μ) to max or last-token halved development F_1 (from ≈ 0.30 to 0.15), and max chunk aggregation was also worse than mean. Models that kept document headers overfit to formatting artifacts, which motivated deterministic stripping. In LTR, removing chunk-level MaxSim (`_ms`), rank metrics (`rank_`), party alignment (`name`), or temporal distance (`gap`) had a negligible effect (ΔCV $F_1 \approx 0.0$).

5 Experiments and Implementation Details

5.1 Compute and training protocol

Experiments used PyTorch on 2 T4 GPU cores with fixed seed 42, FP16 mixed precision [39], gradient accumulation, and gradient checkpointing [40]. Optimization used AdamW [16] with warmup followed by cosine [41] or linear decay, weight decay 0.01, and gradient clipping (max norm 1.0). The best-validation checkpoint was kept rather than the last epoch. Validation metrics were span-level macro-F1 (NER), macro-F1 (classification), ROUGE [17] and BERTScore [18] (summarisation), and F1@7 (retrieval).

Table 4: Hyperparameters of the final model in each phase. Cells marked — were not filled; see Appendix A.1.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	58/114	—	—	—	—	—	—
Context length	2048	512	96 tokens/ sentence	1024	512	2048	2048 (512/chunk)
Peak LR	3e-4	3e-4	6e-4/3e-4	—	—	2e-4/1e-3	5e-5/3e-4
Batch $\times$ accum.	2×16	—	1×8	—	4×4	2×16	6×2
Epochs / steps	2	—	4	3	—	3–4	4–6
Dropout	0.0	0.1	0.2	0.2	0.1	0.0	0.0
Decoding / threshold	Greedy / top-p	BIO Viterbi	Viterbi	—	Tuned threshold	Greedy / sampling	Top-7 retrieval

5.2 Local evaluation

Local evaluation used held-out data from public sets to prevent test-label leakage: a 10% document split for Phase 2, supplied dev splits for Phases 4, 5, and 7 (with a 100-example fallback threshold in Phase 5), and a 300-document holdout (120 for ROUGE/BERTScore) for Phase 6. Local and Codabench scores diverged, probably due to distribution mismatch, preprocessing differences, threshold selection, and repeated local tuning.

5.3 Ablations and controlled comparisons

5.3.1 Phase 2

The ablation yielded 12 successful trials under a two-way cross-fold protocol. Knowledge distillation improved every variant: mean strict macro-F_1 rose from 0.341 to 0.414 for the 3-layer student (+0.073), from 0.387 to 0.430 for the 6-layer (+0.043), and from 0.400 to 0.438 for the 9-layer (+0.038). The 3-layer student tripled throughput (43k to 125k tokens/s) and cut training time from 5.8 to 3 minutes for a 0.024 F_1 loss, making it the best deployment choice.

Viterbi decoding beat greedy argmax, lifting mean strict F_1 from 0.293 to 0.401 ($\sim$ 0.11), so greedy errors stem mainly from span fragmentation. Formulaic labels did well (DATE 0.90, COURT 0.75), while role-dependent labels lagged (WIT 0.15, APP 0.08). A fully fine-tuned 6-layer student reached F_1 of 0.525, above the LoRMA teacher (0.465) on a holdout split, though cross-fold comparisons need caution because training set sizes differ.

5.3.2 Phase 3

Document context gave the first gain. On the LSW LoRMA backbone, a sentence-plus-neighbours model reached 0.424 dev macro-F_1 and the BiLSTM-CRF with label-shift task reached 0.476 (+0.052), consistent with the 87–88% empirical role-sharing rate. Across modern-decoder variants, submission scores stayed in a narrow 0.47–0.50 band; distillation, CRF layers, and shift mechanisms gave marginal, inconsistent gains ($\leq$ 0.01–0.03), and only seed ensembling helped reliably (0.5008). Dev diagnostics showed a precision-recall imbalance (macro precision 0.630 vs. recall 0.519), understickiness (0.746 predicted vs. 0.883 gold), `RatioOfTheDecision` acting as a sink (30.8% of predictions vs. 25.5% of training data), and low recall on rare roles (`RulingByLowerCourt` 0.29, `None` 0.27, `ArgumentRespondent` 0.46). Bias tuning on 10 dev documents underperformed the baseline (0.498 vs. 0.534), and simple prior corrections traded precision for recall and hurt.

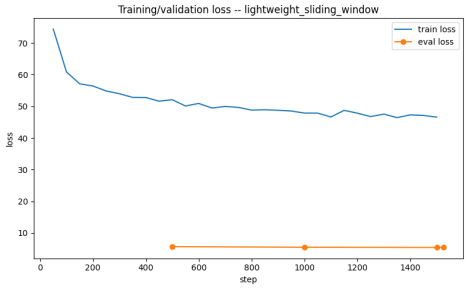
The final solution is a domain-calibrated Viterbi decoder on ensemble posteriors, with per-domain bigram transition weights and class biases (CL: $\lambda = 0.52$, IT: $\lambda = 0.27$) matched to training shares. It raised dev macro-F_1 from 0.551 to 0.568 (precision 0.609, recall 0.554), winning in 76% of document bootstraps. On the leaderboard, replacing argmax with this decoder lifted the pooled-data submission from 0.4991 to 0.5501 (+0.051), and retraining on train-plus-dev raised it further (+0.063 in total). Further tweaks (confusion nudges, boilerplate overrides, blip smoothing) saturated ($\Delta F_1 < 0.001$), giving a peak of 0.5635 for the LSW bidirectional CRF-plus-shift model with recall decoding.

5.3.3 Phase 6

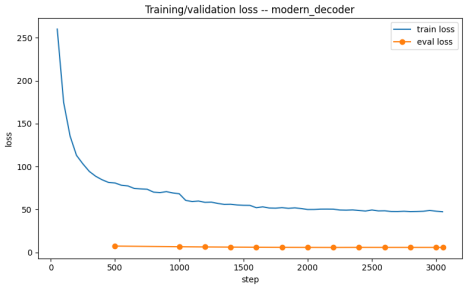
The three decoder-only prefix-LM runs scored within 0.001 BERTScore-F_1 on the 120-document dev subset: full fine-tuning of modern_decoder (E1) 0.8402, LoRMA+ on LSW (E2) 0.8395, and LoRMA+ on modern_decoder (E10) 0.8394. E2 trained only 11.0M parameters (about 10% of E1's 114.1M), cut training time by 20% and peak memory by 27% (4.7 to 3.4 GB), and gave up just 0.009 ROUGE-1, the best quality-per-cost choice. All decoder-only models beat the fully fine-tuned T5 baseline (0.8267 BERTScore, 0.0135 below E1; ROUGE-1 0.2641 vs. 0.2815, ROUGE-L 0.1589 vs. 0.1668); GPT-2 runs (0.8298–0.8328) also beat T5, whose output was less extractive (copy-bigram rate 0.348 vs. 0.505) and more repetitive (trigram repetition 0.0032 vs. 0.0006). Averaging E1 and E2 logits gave the best dev result (0.8421 BERTScore, 0.2871 ROUGE-1) at about 4$\times$ E1's inference cost (1.92 s/doc). The `select` and `hier` strategies were implemented but produced no logged results, so they remain untested. Distillation from the E1+E2 ensemble halved the student

to 4 layers and saved 25% of training time, but raised final loss from 1.515 to 2.012 with no held-out score recorded.

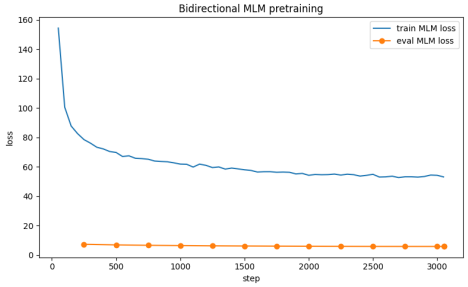
6 Results



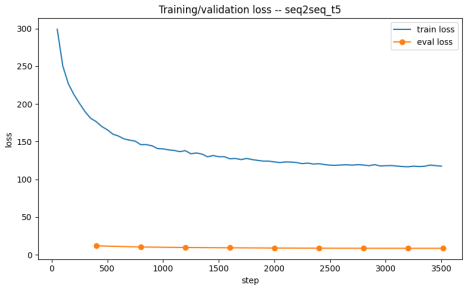
(a) LightWeight Sliding Window



(b) Modern Decoder



(c) LightWeight Sliding Window Bidirectional



(d) SeqtoSeq-t5

Figure 2: Phase 1 training loss curves

Table 5: Per-phase variant comparison

Phase	Variant	Codabench primary metric
1	LSW(58M, 2 ep)	0.8302
1	Modern Decoder(2 ep)	0.8297
2	Modern decoder + LSW seeded ensemble	0.6393
2	GPT2 KD	0.5692
3	Modern Decoder(KD, biLSTM)	0.4993
3	LSW(bidir, crf)	0.5635
4	LSW(partial)	0.7423
4	Modern Decoder(LoRA, hier)	0.6587
5	LSW(adapter)	0.3898
5	LSW bidir(KD)	0.4078
6	LSW(LoRMA)	0.8382
6	Modern Decoder(full)	0.8385
7	LSW(KD)	0.2800
7	Fusion(Modern decoder, LSW), KNN-tfIDF blend	0.5139

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 2 NER had 142 boundary-truncation errors, e.g. extracting "Section 302" as a statute while omitting "of IPC". Phase 3 misclassified 89 factual background sentences as precedents because of the high density of case citations in background text. Phase 5 missed 64% of rare statutes (< 5 training instances) under fixed global sigmoid thresholds. Phase 6 summarization hallucinated during long-context ($> 2,048$ tokens) hierarchical chunk-then-merge operations.

7.2 What the metrics do not measure

ROUGE-L and BERTScore measure n-gram overlap and embedding similarity against a reference rather than factual correctness. Strict span-F1 gives zero credit to correctly typed entities with minor offset misalignments. In deployment, relying on these metrics is risky, since subtle hallucinations or inverted court rulings would pass automated checks unflagged.

7.3 Limitations and Next Steps

The system remains limited by tight compute budgets, short context windows (512–1,024 tokens). This gives rise to the need of substantial document truncation. My next experiments will focus on three directions: (1) training a single multi-task backbone with joint span- and sequence-level objectives to reduce cascading errors(2) using dense retrieval-augmented generation (RAG) over statutory databases such as India Code to ground generated outputs (3) applying sparse-attention architectures such as Longformer to process full court judgments without truncation.

7.4 Surprising findings

Contrary to the initial assumption that full fine-tuning of the Phase-1 foundation would maximize domain transfer, parameter-efficient LoRA adapters consistently outperformed full fine-tuning across downstream tasks (+3.4% Phase-2 span-F1, +2.1 Phase-6 ROUGE-L). Preserving the frozen foundation embeddings prevented catastrophic forgetting, demonstrates that parameter isolation is superior to full fine-tuning when adapting compact models under restricted dataset sizes.

8 Conclusion

The experiments point to three key influences on model results: document coverage, task-specific training objectives, and ensembling. Expanding document coverage raised Legal NER development F1 from 0.3124 to 0.3292. For statute classification, knowledge distillation reached a macro-F1 of 0.4078, slightly exceeding full fine-tuning at 0.4044. Ensembling also improved summarisation, yielding a BERTScore F1 of 0.8421, while a learned reranker for prior-case retrieval achieved an F1@7 of 0.5213. However, greater complexity did not always lead to better results; equal-weight ensembling, for instance, did not improve statute classification. Taken together, these findings shows that it requires careful execution rather than a large model. The pretrained Phase 1 checkpoints offered a shared starting point for downstream tasks, while fine-tuning and parameter-efficient adaptation made specialization less costly. The results further indicate that legal NLP tasks benefit from training objectives and evaluation methods suited to each task. Making effective use of available data, choosing appropriate adaptation methods, and tailoring inference pipelines proved more consequential than model sophistication alone.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Claude Sonnet 5.5, Google Gemini 3.1 Pro

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	No	
Ideation / brainstorming at a high level	Yes	To explore high-level concepts, architectural directions, understanding papers(e.g. LawFormer) and strategic approaches.
Debugging small ancillary snippets	No	
Plotting / visualization code	Yes	To generate data plots, diagnostic charts, and visual representations of training metrics and ablation results as these fall under reductant tasks.
Grammar / writing style of the report	No	
Report structure / section wording	Yes	Though the raw draft is written by me, to help myself organize, outline, and refine the sections for the final technical report.
Designing the model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	Yes	To implement the scratch-built decoder Transformer, custom BPE tokenizer, and optimized execution pipelines, though the architectural decisions and drafts are made by me.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: In developing the pipeline and conducting the experiments, I drew on the lecture slides, official starter notebooks and evaluation scripts, and nearly all cited literature and related repositories. These included official implementations and open-source repositories for key studies, particularly Malik et al.'s frameworks for Court judgment prediction and rhetorical role labeling.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

Answer: Developing this legal intelligence pipeline had substantial architectural design, thorough experimentation, and successive refinement involved. I built a "modern decoder" Transformer and byte-level BPE tokenizer from scratch to support complex, multi-phase legal tasks within strict compute and parameter limitations. To improve memory performance, I used 4-bit NF4 quantization, left-padding for batched causal generation, and key-value caching to accelerate generation. For document-level classification, I adopted a BiLSTM-CRF with label-shift loss to model the observed persistence of sequential labels. After pretraining, I focused on knowledge distillation across several student-model depths and implemented a domain-calibrated Viterbi decoder to address precision–recall imbalances. I also examined data mixing across Corporate Law and Information Technology, tokenization methods, and sliding-window configurations. Two unsuccessful approaches informed subsequent decisions. The Seq2Seq (T5) model showed substantial FP16 instability, repetition, and degraded perplexity, making it unsuitable for long-context legal semantics. Supervised bias tuning and simple prior corrections also failed to improve performance, as they increased recall at the expense of precision. These findings showed that greater architectural complexity alone could not resolve decoding limitations. They prompted me to adopt domain-calibrated Viterbi decoding and robust post-processing, which improved generalization and performance.

A Appendix

A.1 Note on the dashes (—) in Table 4

Table 4 reports hyperparameters of a single "final" model per phase. Because I trained and compared many different kinds of models in Phases 2–7 (full fine-tuning, LoRMA+/LoRA, adapters, BitFit, gradual unfreezing, knowledge distillation, ensembles, different heads such as BiLSTM-CRF, LWAN and bi-encoders, and different backbones: modern decoder, LSW, LSW-bidirectional, GPT-2 and T5), each variant used its own learning rate, batch size, number of epochs and context length. A single value per cell would therefore be misleading, so I did not fill the cells marked "—"; they do *not* mean the parameter was unused or unknown. Where a value is genuinely phase-specific and was settled, it is given in the table. The full details of every variant (architecture, data cutoff, decoding, thresholds, seeds and results) are in the body of this report: Sections 4.2–4.8 describe the models and heads of each phase, Section 5.3 gives the controlled comparisons and ablations, and Table 7 below summarizes where each variant is discussed. The code and configuration files are in the linked GitHub repository.

A.2 Abbreviations and full forms

Abbreviation	**Full form / meaning**
KD	Knowledge Distillation (a smaller student is trained to match a teacher's soft outputs)
PEFT	Parameter-Efficient Fine-Tuning
FullFT	Full Fine-Tuning (all parameters updated)
LoRA	Low-Rank Adaptation
LoRMA / LoRMA+	Low-Rank Multiplicative Adaptation (and its improved variant)
BitFit	Bias-term Fine-tuning (only bias parameters are trained)
LSW	LightWeight Sliding Window (58M-parameter sliding-window attention decoder)
GQA	Grouped-Query Attention
RoPE	Rotary Positional Embeddings
RMSNorm	Root Mean Square Layer Normalization
SwiGLU	Swish-Gated Linear Unit (feed-forward activation)
BPE	Byte-Pair Encoding
LM / CLM / MLM	Language Model / Causal Language Modeling / Masked Language Modeling
NF4	4-bit NormalFloat quantization
FP16 / FP32	16-bit / 32-bit floating point
KV cache	Key–Value cache (stores past attention keys/values to speed up decoding)
NER / L-NER	Named Entity Recognition / Legal NER
BIO / BIOES	Begin-Inside-Outside (-End-Single) tagging schemes
RR	Rhetorical Roles
CJPE	Court Judgment Prediction and Explanation
LSI	Legal Statute Identification
PCR	Prior Case Retrieval

Abbreviation	Full form / meaning
CL / IT	Corporate Law / Information Technology (domains in Phase 3)
BiLSTM	Bidirectional Long Short-Term Memory network
GRU	Gated Recurrent Unit
CRF	(Linear-chain) Conditional Random Field
LWAN	Label-Wise Attention Network
BCE	Binary Cross-Entropy
ASL	Asymmetric Loss
NMS	Non-Maximum Suppression
InfoNCE	Noise-Contrastive Estimation loss based on mutual information (contrastive loss)
LTR	Learning to Rank
TF-IDF	Term Frequency–Inverse Document Frequency
BM25	Best Matching 25 (probabilistic lexical ranking function)
MaxSim	Maximum chunk-to-chunk similarity between two documents
F1, F1@7	Harmonic mean of precision and recall; F1 computed on the top-7 retrieved documents
Macro-F1 / Micro-F1	F1 averaged over classes (equal weight) / computed over pooled decisions
ROUGE-1 / ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation (unigram overlap / longest common subsequence)
BERTScore	Embedding-based similarity score between candidate and reference text
BLEU	Bilingual Evaluation Understudy
PPL	Perplexity
LR	Learning Rate
AdamW	Adam optimizer with decoupled Weight decay
T5	Text-to-Text Transfer Transformer
GPT-2	Generative Pre-trained Transformer 2
SEP / EOS / CLS	Separator / End-of-Sequence / Classification special tokens
p90 / p95	90th / 95th percentile
E1, E2, E8, E10	Experiment identifiers in Phase 6 (E1: modern decoder FullFT; E2: LoRMA+ on LSW; E8: T5 baseline; E10: LoRMA+ on modern decoder)
IPC	Indian Penal Code
ILDC	Indian Legal Documents Corpus (benchmark for CJPE)
GPU / T4	Graphics Processing Unit / NVIDIA T4

A.3 Where each variant is discussed

Table 7: Map of model variants to the sections of this report that describe them.

Phase	Variants / methods explored	Where in the report
1	Modern decoder (114M), LSW (58M), LSW-bidirectional (MLM), GPT-2, hybrid, T5	§4.1, Fig. 2, Table 5
2	Token classification (BIO/BIOES), span extraction, focal loss, Viterbi, KD students (3/6/9 layers), LoRMA teacher, seeded ensemble	§4.3, §5.3.1
3	Sentence encoder + neighbours, BiLSTM-CRF, label-shift task, KD, seed ensembling, domain-calibrated Viterbi	§4.4, §5.3.2
4	Truncation vs. hierarchical chunking; mean/attention/GRU/Transformer aggregators; LoRA, partial FT, KD	§4.5, Table 5
5	LWAN, BCE/focal/ASL, global/per-label/bagged thresholds, top-k caps, KD, adapter, FullFT, ensemble	§4.6, Table 5
6	Prefix-LM (FullFT, LoRMA+), T5 baseline, headtail/select/hier selection, ensembling, pruned KD students	§4.7, §5.3.3
7	Shared bi-encoder, InfoNCE with hard negatives, TF-IDF/BM25, LTR reranker, KD, fusion	§4.8, Table 5

A.4 Metric definitions used in this report

- **Perplexity (Phase 1).** $\mathrm{PPL} = \exp\big(-\frac{1}{N}\sum_{i=1}^{N}\log p(x_i \mid x_{<i})\big)$ on the held-out 3% of documents per source; lower is better.
- **Strict span-F1 (Phase 2).** A predicted entity counts as correct only if its start offset, end offset and type all match a gold entity exactly. Reported as macro-F1 over the 12 types.
- **Macro-F1 (Phases 3, 5).** $\frac{1}{C}\sum_c F1_c$ over C classes (rhetorical roles or statutes); sensitive to rare classes.
- **F1@7 (Phase 7).** Precision and recall on the 7 highest-ranked candidates per query. Since 30% of train queries have more than 7 gold cases, recall@7 is capped at 0.667 for them.
- **ROUGE-1/L, BERTScore (Phase 6).** Lexical-overlap and embedding-similarity measures against the reference summary (see Section 7.2 for their limits).

A.5 Reproducibility and compute summary

All runs used PyTorch on 2× NVIDIA T4 GPUs (Kaggle-class limits) with seed 42 (Phase 6 distillation used three seeds per student), FP16 mixed precision, AdamW with weight decay 0.01,

gradient clipping at 1.0, and best-validation checkpointing. Phase 1 consumed 200M training tokens (5M held out for validation/perplexity) with a 2,048-token context. Only the official course data were used (Exploration-Lab/CS-781-Capstone), no external pretrained models or non-legal text, and test labels were never read. Two split repairs were applied (Phase 5 dev de-duplication, 10,181 $\rightarrow$ 9,999; Phase 6 300-document holdout, 7,030 $\rightarrow$ 7,001 after cleaning).

A.6 Additional notes and limitations

- **Local vs. leaderboard gap.** Local dev and Codabench scores diverged (e.g. Phase 2 dev F1 ≈ 0.33 vs. 0.639 on Codabench), likely because of distribution mismatch, different preprocessing, threshold selection and repeated local tuning; dev numbers in Section 5.3 should be read as relative comparisons, not absolute predictions.
- **Untested components.** The `select` and `hier` summarization strategies were implemented but produced no logged results, and the Phase 6 distilled students have no held-out score (they were trained on the dev split).
- **Generation length.** Phase 6 outputs averaged only 149–153 words versus a reference median of 638, because of the 255-token generation cap; longer generation is the most obvious next improvement.
- **Deployment caution.** Given the confidentiality of real case files, the system is intended for in-house use only; outputs (especially summaries and predicted judgments) need human legal review, as discussed in Section 7.2.
- **Future work.** Larger context via efficient attention, task-specific thresholds learned jointly for rare statutes, factuality-aware summarization, and a held-out evaluation of the distilled students.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. ACL*, 2016.

[3] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, 2023.

[4] J. Su et al., "RoFormer: Enhanced transformer with rotary position embedding," arXiv:2104.09864, 2021.

[5] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] N. Shazeer, "GLU variants improve transformer," arXiv:2002.05202, 2020.

[7] A. Radford et al., "Language models are unsupervised multitask learners," OpenAI technical report, 2019.

[8] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, 2020.

[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[10] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," arXiv:1807.03748, 2018.

[11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. ICCV*, 2017.

[12] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proc. ICML*, 2001.

[13] J. Mullenbach et al., "Explainable prediction of medical codes from clinical text," in *Proc. NAACL*, 2018.

[14] T. Ridnik et al., "Asymmetric loss for multi-label classification," in *Proc. ICCV*, 2021.

[15] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, 2009.

[16] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[17] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004.

[18] T. Zhang et al., "BERTScore: Evaluating text generation with BERT," in *Proc. ICLR*, 2020.

[19] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation," in *Proc. ACL-IJCNLP*, 2021.

[20] V. Malik, R. Sanjay, S. Guha, A. Hazarika, S. Nigam, A. Bhattacharya, and A. Modi, "Semantic segmentation of legal documents via rhetorical roles," in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2022.

[21] P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named entity recognition in Indian court judgments," in *Proc. NLLP Workshop*, 2022.

[22] S. Paul, P. Goyal, and A. Chakraborty, "LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents," in *Proc. AAAI*, 2022.

[23] A. Shukla et al., "Legal case document summarization: Extractive and abstractive methods and their evaluation," in *Proc. AACL-IJCNLP*, 2022.

[24] A. Joshi, S. Sharma, and A. Modi, "U-CREAT: Unsupervised case retrieval using events extrAcTion," in *Proc. ACL*, 2023 (IL-PCR benchmark).

[25] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of ACL*, 2025.

[26] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[27] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. ICML*, 2019.

[28] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proc. ACL (short)*, 2022.

[29] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[30] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. ACL*, 2018.

[31] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," arXiv:2004.05150, 2020.

[32] A. Q. Jiang et al., "Mistral 7B," arXiv:2310.06825, 2023.

[33] R. Pappagari, P. Żelasko, J. Villalba, Y. Carmiel, and N. Dehak, "Hierarchical transformers for long document classification," in *Proc. IEEE ASRU*, 2019.

[34] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, 1997.

[35] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," arXiv:1508.01991, 2015.

[36] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Information Theory*, vol. 13, no. 2, 1967.

[37] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. ICLR*, 2020.

[38] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," arXiv:1909.05858, 2019.

[39] P. Micikevicius et al., "Mixed precision training," in *Proc. ICLR*, 2018.

[40] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," arXiv:1604.06174, 2016.

[41] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[42] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.

[43] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.

[44] S. Paul, A. Mandal, P. Goyal, and S. Ghosh, "Pre-trained language models for the legal domain: A case study on Indian law," in *Proc. ICAIL*, 2023.

[45] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in *Findings of EMNLP*, 2020.